

9.5 Adaptable Priority Queues

The methods of the priority queue ADT given in Section 9.1.2 are sufficient for most basic applications of priority queues, such as sorting. However, there are situations in which additional methods would be useful, as shown by the scenarios below involving the standby airline passenger application.

- A standby passenger with a pessimistic attitude may become tired of waiting and decide to leave ahead of the boarding time, requesting to be removed from the waiting list. Thus, we would like to remove from the priority queue the entry associated with this passenger. Operation `removeMin` does not suffice since the passenger leaving does not necessarily have first priority. Instead, we want a new operation, `remove`, that removes an arbitrary entry.
- Another standby passenger finds her gold frequent-flyer card and shows it to the agent. Thus, her priority has to be modified accordingly. To achieve this change of priority, we would like to have a new operation `replaceKey` allowing us to replace the key of an existing entry with a new key.
- Finally, a third standby passenger notices her name is misspelled on the ticket and asks it to be corrected. To perform this change, we need to update the passenger's record. Hence, we would like to have a new operation `replaceValue`, allowing us to replace the value of an existing entry with a new value.

The Adaptable Priority Queue ADT

The above scenarios motivate the definition of a new *adaptable priority queue* ADT that extends the priority queue ADT with additional functionality. We will see another application of adaptable priority queues when implementing certain graph algorithms in Sections 14.6.2 and 14.7.1.

In order to implement methods `remove`, `replaceKey`, and `replaceValue` efficiently, we need a mechanism for finding a user's element within a priority queue, ideally in a way that avoids performing a linear search through the entire collection. In the original definition of the priority queue ADT, a call to `insert( $k, v$ )` formally returns an instance of type `Entry` to the user. In order to be able to update or remove an entry in our new adaptable priority queue ADT, the user must retain that `Entry` object as a token that can be sent back as a parameter to identify the relevant entry. Formally, the adaptable priority queue ADT includes the following methods (in addition to those of the standard priority queue):

`remove( $e$ )`: Removes entry e from the priority queue.

`replaceKey( $e, k$ )`: Replaces the key of existing entry e with k .

`replaceValue( $e, v$ )`: Replaces the value of existing entry e with v .

An error occurs with each of these methods if parameter e is invalid (for example, because it had previously been removed from the priority queue).

9.5.1 Location-Aware Entries

To allow an entry instance to encode a location within a priority queue, we extend the `PQEntry` class (originally defined with the `AbstractPriorityQueue` base class), adding a third field that designates the current index of an entry within the array-based representation of the heap, as shown in Figure 9.10. (This approach is similar to our recommendation, on page 281, for implementing the positional list abstraction with an array.)

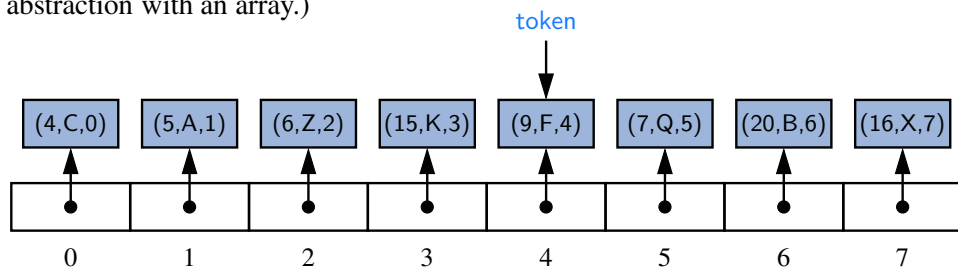


Figure 9.10: Representing a heap using an array of location-aware entries. The third field of each entry instance corresponds to the index of that entry within the array. Identifier token is presumed to be an entry reference in the user’s scope.

When we perform priority queue operations on our heap, causing entries to be relocated within our structure, we must make sure to update the third field of each affected entry to reflect its new index within the array. As an example, Figure 9.11 shows the state of the above heap after a call to `removeMin()`. The heap operation causes the minimal entry, (4,C), to be removed, and the last entry, (16,X), to be temporarily moved from the last position to the root, followed by a down-heap bubble phase. During the down-heap, element (16,X) is swapped with its left child, (5,A), at index 1 of the list, then swapped with its right child, (9,F), at index 4 of the list. In the final configuration, the last field for all affected entries has been modified to reflect their new location.

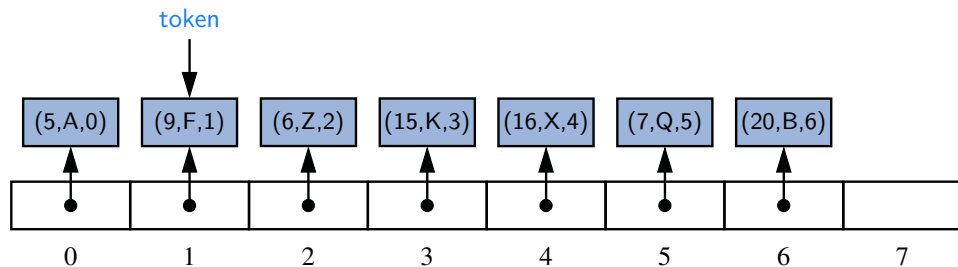


Figure 9.11: The result of a call to `removeMin()` on the heap originally portrayed in Figure 9.10. Identifier token continues to reference the same entry as in the original configuration, but the placement of that entry in the array has changed, as has the third field of the entry.

9.5.2 Implementing an Adaptable Priority Queue

Code Fragments 9.12 and 9.13 present a Java implementation of an adaptable priority queue, as a subclass of the `HeapPriorityQueue` class from Section 9.3.2. We begin by defining a nested `AdaptablePQEntry` class (lines 5–15) that extends the inherited `PQEntry` class, augmenting it with an additional index field. The inherited `insert` method is overridden, so that we create and initialize an instance of the `AdaptablePQEntry` class (not the original `PQEntry` class).

An important aspect of our design is that the original `HeapPriorityQueue` class relies exclusively on a protected `swap` method for all low-level data movement during up-heap or down-heap operations. The `AdaptablePriorityQueue` class overrides that utility in order to update the stored indices of our location-aware entries when they are relocated (as discussed on the previous page).

When an entry is sent as a parameter to `remove`, `replaceKey`, or `replaceValue`, we rely on the new index field of that entry to designate where the element resides in the heap (a fact that is easily validated). When a key of an existing entry is replaced, that new key may violate the heap-order property by being either too big or too small. We provide a new bubble utility that determines whether an up-heap or down-heap bubbling step is warranted. When removing an arbitrary entry, we replace it with the last entry in the heap (to maintain the complete binary tree property) and perform the bubbling step, since the displaced element may have a key that is too large or too small for its new location.

Performance of Adaptable Priority Queue Implementations

The performance of an adaptable priority queue by means of our location-aware heap structure is summarized in Table 9.5. The new class provides the same asymptotic efficiency and space usage as the nonadaptive version, and provides logarithmic performance for the new locator-based `remove` and `replaceKey` methods, and constant-time performance for the new `replaceValue` method.

Method	Running Time
size, isEmpty, min	$O(1)$
insert	$O(\log n)$
remove	$O(\log n)$
removeMin	$O(\log n)$
replaceKey	$O(\log n)$
replaceValue	$O(1)$

Table 9.5: Running times of the methods of an adaptable priority queue with size n , realized by means of our array-based heap representation. The space requirement is $O(n)$.

```

1  /** An implementation of an adaptable priority queue using an array-based heap. */
2  public class HeapAdaptablePriorityQueue<K,V> extends HeapPriorityQueue<K,V>
3              implements AdaptablePriorityQueue<K,V> {
4
5      //----- nested AdaptablePQEntry class -----
6      /** Extension of the PQEntry to include location information. */
7      protected static class AdaptablePQEntry<K,V> extends PQEntry<K,V> {
8          private int index; // entry's current index within the heap
9          public AdaptablePQEntry(K key, V value, int j) {
10             super(key, value); // this sets the key and value
11             index = j; // this sets the new field
12         }
13         public int getIndex() { return index; }
14         public void setIndex(int j) { index = j; }
15     } //----- end of nested AdaptablePQEntry class -----
16
17     /** Creates an empty adaptable priority queue using natural ordering of keys. */
18     public HeapAdaptablePriorityQueue() { super(); }
19     /** Creates an empty adaptable priority queue using the given comparator. */
20     public HeapAdaptablePriorityQueue(Comparator<K> comp) { super(comp); }
21
22     // protected utilities
23     /** Validates an entry to ensure it is location-aware. */
24     protected AdaptablePQEntry<K,V> validate(Entry<K,V> entry)
25         throws IllegalArgumentException {
26         if (!(entry instanceof AdaptablePQEntry))
27             throw new IllegalArgumentException("Invalid entry");
28         AdaptablePQEntry<K,V> locator = (AdaptablePQEntry<K,V>) entry; // safe
29         int j = locator.getIndex();
30         if (j >= heap.size() || heap.get(j) != locator)
31             throw new IllegalArgumentException("Invalid entry");
32         return locator;
33     }
34
35     /** Exchanges the entries at indices i and j of the array list. */
36     protected void swap(int i, int j) {
37         super.swap(i,j); // perform the swap
38         ((AdaptablePQEntry<K,V>) heap.get(i)).setIndex(i); // reset entry's index
39         ((AdaptablePQEntry<K,V>) heap.get(j)).setIndex(j); // reset entry's index
40     }

```

Code Fragment 9.12: An implementation of an adaptable priority queue. (Continues in Code Fragment 9.13.) This extends the `HeapPriorityQueue` class of Code Fragments 9.8 and 9.9.

```

41  /** Restores the heap property by moving the entry at index j upward/downward.*/
42  protected void bubble(int j) {
43      if (j > 0 && compare(heap.get(j), heap.get(parent(j))) < 0)
44          upheap(j);
45      else
46          downheap(j);                      // although it might not need to move
47  }
48
49  /** Inserts a key-value pair and returns the entry created. */
50  public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
51      checkKey(key);                        // might throw an exception
52      Entry<K,V> newest = new AdaptablePQEntry<>(key, value, heap.size());
53      heap.add(newest);                     // add to the end of the list
54      upheap(heap.size() - 1);             // upheap newly added entry
55      return newest;
56  }
57
58  /** Removes the given entry from the priority queue. */
59  public void remove(Entry<K,V> entry) throws IllegalArgumentException {
60      AdaptablePQEntry<K,V> locator = validate(entry);
61      int j = locator.getIndex();
62      if (j == heap.size() - 1)              // entry is at last position
63          heap.remove(heap.size() - 1);     // so just remove it
64      else {
65          swap(j, heap.size() - 1);         // swap entry to last position
66          heap.remove(heap.size() - 1);     // then remove it
67          bubble(j);                       // and fix entry displaced by the swap
68      }
69  }
70
71  /** Replaces the key of an entry. */
72  public void replaceKey(Entry<K,V> entry, K key)
73      throws IllegalArgumentException {
74      AdaptablePQEntry<K,V> locator = validate(entry);
75      checkKey(key);                       // might throw an exception
76      locator.setKey(key);                 // method inherited from PQEntry
77      bubble(locator.getIndex());          // with new key, may need to move entry
78  }
79
80  /** Replaces the value of an entry. */
81  public void replaceValue(Entry<K,V> entry, V value)
82      throws IllegalArgumentException {
83      AdaptablePQEntry<K,V> locator = validate(entry);
84      locator.setValue(value);             // method inherited from PQEntry
85  }

```

Code Fragment 9.13: An implementation of an adaptable priority queue (continued from Code Fragment 9.12).